# DESIGN AND IMPLEMENTATION OF A WEB-BASED GPS LOCATION TRACKING AND MANAGEMENT SYSTEM FOR NYANZA DISTRICT

**A final-year thesis submitted to the University of Lay Adventists of Kigali (UNILAK) in partial fulfilment of the requirements for the award of the Bachelor of Science degree in Computing and Information Sciences**

**Student:** [Student Name]
**Registration Number:** [Registration Number]
**Supervisor:** [Supervisor Name]
**Campus:** Nyanza Campus
**Academic Year:** [Academic Year]

---

# DEDICATION

This work is dedicated to my parents, family members, lecturers, and friends whose encouragement, patience, and moral support enabled me to complete this academic project. It is also dedicated to students and public-service practitioners who seek practical and affordable ways of using information technology to improve the management of mobile activities in their communities.

# DECLARATION

I, **[Student Name]**, declare that this thesis entitled **“Design and Implementation of a Web-Based GPS Location Tracking and Management System for Nyanza District”** is my original work and has not been submitted to any other institution for the award of an academic qualification. All sources used in this work have been acknowledged through appropriate citations and references.

**Student signature:** _____
**Date:** _____

# ACKNOWLEDGMENT

I thank God for the strength and guidance that enabled me to complete this study. I sincerely thank my supervisor, **[Supervisor Name]**, for the academic direction, constructive comments, and continuous support provided during the development of this thesis. I am grateful to the lecturers and staff of the University of Lay Adventists of Kigali for the knowledge and skills acquired throughout my programme.

I also appreciate the support of my classmates, friends, and family members. Their encouragement assisted me during the requirements analysis, implementation, testing, and preparation of this document. Finally, I acknowledge the public information made available by Nyanza District and the Government of Rwanda, which helped to establish the institutional and geographical context of this study.

# TABLE OF CONTENT

**Front matter**

- Dedication
- Declaration
- Acknowledgment
- Table of Content
- List of Figures
- List of Symbols & Abbreviations
- Abstract

**Chapter 1: Introduction**

- 1.1 Background to the study
- 1.2 Problem statement
- 1.3 Objectives
  - 1.3.1 General objective
  - 1.3.2 Specific objectives
- 1.4 Significance
  - 1.4.1 Personal
  - 1.4.2 Institutional
  - 1.4.3 Public interest

- 1.5 Scope of the study
- 1.6 Expected results
- 1.7 Organization of the study

**Chapter 2: Literature Review**

- 2.1 Introduction
- 2.2 Definitions
  - 2.2.1 Key concepts of the topic
  - 2.2.2 IT concepts
- 2.3 Comparative study of similar systems
- 2.4 Personal contribution

**Chapter 3: Analysis of Existing System**

- 3.1 Introduction
- 3.2 Organization overview
- 3.3 Vision, mission and development objectives
- 3.4 Description of existing system activities
  - 3.4.1 Tracker registration
  - 3.4.2 Location reporting
  - 3.4.3 Trip monitoring
  - 3.4.4 Trip reporting and record keeping
- 3.5 Weaknesses of the existing system
- 3.6 Proposed solutions

**Chapter 4: Analysis & Design of New System**

- 4.1 Introduction
- 4.2 Development methodologies, models and tools
- 4.3 Function diagram
- 4.4 Context diagram
- 4.5 Data flow diagram (level 0)
- 4.6 Functional and non-functional requirements
- 4.7 Database design

    - 4.7.1 Entities
    - 4.7.2 Relationships and cardinality
    - 4.7.3 Attributes
    - 4.7.4 Physical model
    - 4.7.5 Logical model
    - 4.7.6 Data dictionary

**Chapter 5: Implementation of the New System**

- 5.1 Introduction
- 5.2 Implementation environment
- 5.3 Server-side implementation
- 5.4 REST API implementation
- 5.5 Database implementation
- 5.6 Dashboard implementation
- 5.7 Smartphone tracker implementation
- 5.8 Map and route implementation
- 5.9 Application workflow
- 5.10 Deployment and operation
- 5.11 Implementation summary

**Chapter 6: References**


# LIST OF FIGURES


- **Figure 3.1:** Existing tracker registration process
- **Figure 3.2:** Existing location reporting process
- **Figure 3.3:** Existing trip monitoring process
- **Figure 3.4:** Existing trip reporting and record-keeping process
- **Figure 4.1:** Function diagram of the proposed GPS management system
- **Figure 4.2:** Context diagram of the proposed GPS management system
- **Figure 4.3:** Level 0 data flow diagram
- **Figure 4.4:** Entity relationship diagram

# LIST OF SYMBOLS & ABBREVIATIONS

| Symbol or abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| BSc | Bachelor of Science |
| DBMS | Database Management System |
| DFD | Data Flow Diagram |
| ERD | Entity Relationship Diagram |
| GPS | Global Positioning System |
| HTML | HyperText Markup Language |
| HTTP | HyperText Transfer Protocol |
| HTTPS | HyperText Transfer Protocol Secure |
| JSON | JavaScript Object Notation |
| NISR | National Institute of Statistics of Rwanda |
| OOM | Object-Oriented Methodology |
| REST | Representational State Transfer |
| SQL | Structured Query Language |
| UNILAK | University of Lay Adventists of Kigali |
| UML | Unified Modeling Language |

# ABSTRACT

This study presents the design and implementation of a web-based GPS location tracking and management system for Nyanza District. The study responds to the difficulty of obtaining timely, centralized, and reliable information about the location and movement of field personnel, mobile teams, or vehicles when monitoring depends on telephone calls, informal messages, and manually maintained records. The proposed system uses a smartphone browser as a low-cost GPS tracking

device. The browser requests location permission, captures latitude, longitude, speed, accuracy, and time, and sends the data to a central web server. An administrator accesses a web dashboard to register trackers, view the latest locations on an interactive map, inspect location history, and manage trips.

The system was designed using object-oriented analysis principles and UML-oriented modelling. The implementation uses HTML, CSS, and JavaScript for the client interface, Node.js and Express for the application server, SQLite for relational data storage, and Leaflet with OpenStreetMap for map visualization. The source code, release package, and implementation documentation are maintained in the project repository at https://github.com/MiracleCodez/gps-system. The database contains devices, locations, and trips. The system is intended as a practical, deployable solution that can be used and demonstrated without owning a car or purchasing a dedicated GPS unit, because a smartphone carried by a walking user can simulate the movement of a vehicle or field team.

The expected contribution is a transparent, affordable, and extensible platform for location monitoring. The system is not presented as a replacement for a production fleet-management platform without further work. Production deployment would require authenticated users, role-based access, HTTPS, stronger privacy controls, retention policies, and testing with district stakeholders. The completed system establishes the technical basis for these future extensions.

**Keywords:** GPS, location tracking, fleet management, smartphone geolocation, web application, Nyanza District, information system.

# CHAPTER 1: INTRODUCTION

## 1.1 Background to the study

The management of mobile activities depends on knowing where people, vehicles, or field teams are located and whether they are following planned activities. In a district environment, such information can support coordination of field visits, inspection activities, public-service operations, transport, emergency response, and other tasks that require movement between locations. When location information is available only through telephone calls or informal messages, managers cannot easily obtain a shared and historical view of activity.

A Global Positioning System (GPS) receiver determines a device’s position by processing signals from navigation satellites. Modern smartphones combine GPS capability with mobile connectivity and web browsers. A smartphone can therefore collect location coordinates and transmit them to a server without the installation of a dedicated vehicle tracker. This approach is suitable for the implemented system because it reduces hardware cost and allows a walking user to simulate the movement of a vehicle or field worker.

Nyanza District is one of the eight districts in Rwanda's Southern Province. The official district profile describes it as a historical and cultural centre of Rwanda and states that it is composed of 10 sectors, 51 cells, and 420 villages. The district's economic activities include agriculture, livestock, trade, and tourism [1]. These administrative and economic characteristics create a context in which timely information about mobile activities can be valuable for coordination and service delivery.

The transport sector is also relevant to the study. Rwanda's Ministry of Infrastructure identifies transport responsibilities that include government fleet management and traffic management. The ministry further emphasizes mobility, accessibility, reliability, safety, monitoring, and evaluation of transport services [2]. A small web-based tracking system cannot address all national transport requirements, but it can demonstrate how digital location records may support monitoring and reporting at an operational level.

The study is academically conducted at the University of Lay Adventists of Kigali. UNILAK's official profile states that the university has a Nyanza Campus and that its mission includes education, scientific and technological research, and community service [3]. The proposed system is therefore aligned with a computing research project that applies software development skills to a practical district-level problem.

## 1.2 Problem statement

The problem addressed by this study is the absence of a simple and affordable platform for centralized GPS location monitoring in the selected Nyanza District context. In a manual or decentralized arrangement, a field user may communicate a location by telephone or message, while a supervisor records the information in a notebook, spreadsheet, or separate communication thread. Such an arrangement does not provide a continuously updated map, consistent timestamps, automated history, or a reliable method of comparing activities over time.

The absence of centralized location records can lead to delayed decision-making, incomplete trip histories, difficulty verifying reported movements, and additional administrative work. It also limits the ability of a manager to view several trackers from one interface. Dedicated vehicle GPS systems may solve some of these problems, but they introduce hardware, installation, subscription, and maintenance costs that are not appropriate for an initial BSc implementation.

This study therefore proposes a smartphone-based web system that captures GPS coordinates, stores them in a relational database, and displays the information on a management dashboard. The project uses a smartphone as the tracking device, so the researcher does not need to own a car or dedicated tracking hardware to demonstrate the system.

## 1.3 Objectives

### 1.3.1 General objective

To design and implement a web-based GPS location tracking and management system for Nyanza District.

### 1.3.2 Specific objectives

1. To examine the existing methods used for registering, communicating, monitoring, and recording mobile activities in the selected study context.
2. To identify the functional and non-functional requirements of a smartphone-based GPS management system.
3. To design the system architecture, process models, database structure, and user interfaces of the proposed system.
4. To implement a smartphone tracking interface that requests location permission and transmits GPS coordinates to a web server.
5. To implement an administrative dashboard for tracker registration, map-based monitoring, telemetry inspection, and route-history review.
6. To implement a relational database for storing trackers, location points, and trip records.
7. To test the system using functional, integration, validation, and usability-oriented test cases.

## 1.4 Significance

### 1.4.1 Personal

The study enables the researcher to apply software engineering knowledge to a complete information-system project. It provides practical experience in requirements analysis, object-oriented modelling, REST API development, database design, web programming, browser geolocation, map visualization, testing, and technical documentation. The project also creates a foundation for future research into geofencing, route optimization, spatial databases, and mobile applications.

### 1.4.2 Institutional

For UNILAK, the project provides an applied computing case study connected to community service and technological research. It may support teaching and demonstration in systems analysis, web development, database management, and information systems. The project can also be extended by future students who wish to add authentication, PostgreSQL/PostGIS, analytics, notifications, or dedicated GPS-device integration.

### 1.4.3 Public interest

A low-cost location-management system may be useful to public institutions, small transport operators, delivery teams, non-governmental organizations, and community-service programmes that cannot immediately purchase dedicated tracking infrastructure. The system may improve visibility of mobile activities and reduce dependence on fragmented communication. However, location data is sensitive; public deployment must therefore include consent, access control, data minimization, secure transmission, and clear retention rules.

## 1.5 Scope of the study

The geographical context of the study is Nyanza District in Rwanda. The academic and development context is UNILAK, particularly the Nyanza Campus and the computing discipline. The system scope is a web-based GPS location tracking and management system.

Functionally, the system covers tracker registration, smartphone geolocation permission, transmission of latitude and longitude, storage of speed and accuracy, latest-location display, tracker status, trip start and end, and route-history review. The system includes a dashboard for an administrator or fleet manager and a phone tracker page for the field user.

The system does not include vehicle engine diagnostics, fuel sensors, dedicated GPS hardware, automatic emergency dispatch, payment processing, facial recognition, national-scale public transport planning, or production-grade multi-tenant security. These areas are identified as possible future extensions rather than current deliverables.

## 1.6 Expected results

The study is expected to produce a working web application through which a smartphone can act as a GPS tracking device. The system should allow an administrator to register a tracker and view the tracker’s latest position on an interactive map. It should store valid location points in a relational database and provide a history that can be reviewed after movement has taken place.

The study is also expected to produce a documented system design comprising requirements, process diagrams, a context diagram, a level 0 data-flow diagram, a function diagram, an entity relationship diagram, a logical data model, a physical database model, and a data dictionary. Finally, the study should provide test evidence showing whether the implemented functions satisfy the specified requirements.

## 1.7 Organization of the study

Chapter One introduces the study by presenting its background, problem, objectives, significance, scope, expected results, and organization. Chapter Two reviews key concepts and compares similar categories of systems. Chapter Three analyses the existing processes considered in the

study context, identifies their weaknesses, and proposes solutions. Chapter Four presents the development methodology and the design of the proposed system, including the function diagram, context diagram, data-flow diagram, requirements, and database design.

# CHAPTER 2: LITERATURE REVIEW

## 2.1 Introduction

This chapter reviews concepts related to GPS location tracking, district administration, mobility management, and information systems. It defines the main terms used in the study, explains the information-technology concepts that support the proposed solution, compares similar system categories, and identifies the contribution of the present study.

## 2.2 Definitions

### 2.2.1 Key concepts of the topic

**District.** A district is a local administrative area established to organize public administration and the delivery of services. In this study, Nyanza District refers to the district in Rwanda's Southern Province that includes sectors, cells, and villages under a decentralized administrative structure. The official district profile states that Nyanza has 10 sectors, 51 cells, and 420 villages [1].

**GPS.** The Global Positioning System is a satellite-based positioning system that enables a receiver to estimate its geographical position. In this study, GPS refers to the location capability available on a smartphone and accessed through the browser's geolocation interface.

**Location coordinate.** A location coordinate is a numeric representation of position, normally expressed as latitude and longitude. The system also records time, speed, and estimated accuracy for each coordinate observation.

**Tracker.** A tracker is a registered device that submits location information to the system. In the implemented system, the tracker is normally a smartphone browser. The design also allows the concept to represent a future dedicated GPS device.

**Fleet management.** Fleet management is the coordination, monitoring, and administration of vehicles or mobile assets. In this study, the term is used broadly to include the management of registered trackers, trips, positions, and operational history. The implemented system is not limited to vehicles and can represent field workers or other mobile teams.

**Trip.** A trip is a period of movement that begins when the user selects "Start trip" and ends when the user selects "End trip". A trip is associated with one tracker and may contain many location records.

**Geofencing.** Geofencing is the creation of a virtual geographic boundary that can trigger an event when a tracker enters or leaves an area. Geofencing is not implemented in the current MVP but is a relevant future extension.

**Route history.** Route history is the ordered collection of location points recorded for a tracker. The dashboard displays these points as a table and can connect them as a route line on the map.

**Real-time monitoring.** Real-time monitoring is the observation of recent system events with a short delay between data collection and display. In this implemented system, the tracker sends updates when the browser reports a new position and the dashboard refreshes recent device data periodically.

### 2.2.2 IT concepts

**Information.** Information is data that has been processed or organized so that it has meaning for a user. A latitude value alone is data; a map marker labelled with the tracker name, time, speed, and status is information useful for management.

**Software.** Software is a collection of programs, configuration, and related files that instruct a computing device to perform tasks. The GPS system includes server software, browser-side scripts, stylesheets, database initialization logic, and configuration files.

**System.** A system is a set of connected components that interact to achieve a purpose. The proposed system contains users, a smartphone tracker, browser pages, an API, validation logic, a database, and a map interface.

**Information system.** An information system is an organized combination of people, procedures, data, hardware, software, and communication resources used to collect, process, store, and distribute information. The proposed GPS system is an information system because it transforms location observations into operational information for a manager.

**Database.** A database is an organized collection of related data that can be stored, queried, updated, and protected. The implemented system uses SQLite to store devices, locations, and trips.

**Application Programming Interface.** An API is a defined interface through which software components exchange requests and responses. The system’s REST API receives location submissions and returns device, location, and trip information as JSON.

**Web application.** A web application is software accessed through a web browser and supported by server-side processing. The proposed system has a dashboard page and a phone tracker page served by the Express web server.

**User interface.** A user interface is the visible and interactive part of a system through which a person performs tasks. The dashboard provides map, device, telemetry, and history controls, while the tracker page provides tracking and trip controls.

## 2.3 Comparative study of similar systems

The proposed system can be compared with four broad categories of existing solutions. Dedicated vehicle GPS platforms usually provide continuous tracking, hardware integration, geofencing, driver behaviour analysis, and alerts. Their main disadvantage for this study is the cost and complexity of purchasing, installing, and maintaining dedicated hardware.

Mobile fleet applications use smartphone location services and often provide route monitoring, driver communication, proof of delivery, and reports. They are closer to the proposed solution because they use a phone as the tracking device. However, many commercial applications are subscription-based and may include features that are unnecessary for a small BSc implementation.

Manual reporting systems use telephone calls, messaging applications, paper forms, or spreadsheets. They require little technical infrastructure, but they do not provide consistent real-time visibility, automatic coordinate validation, or a unified route history. They also create additional work when a supervisor needs to compare multiple activities.

Public mapping platforms provide map visualization and routing but do not necessarily provide a complete management workflow. The proposed system combines the map interface with tracker registration, location persistence, trip records, and an administrative dashboard. Its distinctive contribution is therefore not the invention of GPS or maps; it is the integration of accessible smartphone geolocation with a small, understandable management system that can be demonstrated without a vehicle.

| System category | Tracking device | Central history | Map dashboard | Hardware cost | Suitability for this study |
|---|---|---|---|---|---|
| Dedicated fleet platform | Installed GPS unit | Yes | Yes | High | Useful as a reference, costly for this implementation |
| Commercial mobile fleet app | Smartphone | Yes | Usually | Subscription | Similar concept, less transparent for study |
| Manual calls and spreadsheets | Phone or paper | Fragmented | No | Low | Current-problem reference |
| Proposed GPS system | Smartphone browser | Yes | Yes | Low | Appropriate for the BSc system |

The comparison indicates that the proposed system occupies a practical middle ground. It provides more structure than manual reporting while avoiding the hardware requirements of a dedicated fleet platform. The implemented system should not be evaluated as equivalent to mature commercial products in uptime, security, or large-scale analytics.

## 2.4 Personal contribution

The personal contribution of this study is the design and implementation of an end-to-end smartphone-based GPS information system adapted to the Nyanza District context. The work combines requirements analysis, process modelling, object-oriented thinking, database design, REST API implementation, browser geolocation, and map visualization in one coherent implemented system.

The study also contributes a low-cost demonstration method. Because the smartphone itself acts as the tracking device, the researcher can walk with the phone and generate a realistic route without owning a car. This makes the system accessible for academic demonstration while preserving the essential data flow of a vehicle or field-team tracking system.

# CHAPTER 3: ANALYSIS OF EXISTING SYSTEM

## 3.1 Introduction

This chapter analyses the existing process considered by the study before the introduction of the proposed web-based system. Since no formal district workflow records or interview transcripts were supplied for this draft, the existing activities are modelled as a manual or semi-manual location-reporting arrangement. Before final submission, the researcher should validate these descriptions through interviews, observation, questionnaires, or official process documents and replace any assumption that does not match the selected department or organization.

The analysis focuses on four activities: tracker registration, location reporting, trip monitoring, and trip reporting. Each activity is described with a process figure so that the transition from the existing arrangement to the proposed system is clear.

## 3.2 Organization overview

Nyanza District is located in Rwanda’s Southern Province and is one of the province’s eight districts. According to its official profile, the district is historically and culturally significant, contains 10 sectors, 51 cells, and 420 villages, and has economic activities that include agriculture, livestock,

trade, and tourism [1]. These characteristics mean that district operations may involve coordination across multiple administrative and physical locations.

The academic host institution for this study is the University of Lay Adventists of Kigali. UNILAK's official profile states that the university was established by the Federation of Adventist Parents Associations for the Development of Education in Rwanda and that it operates campuses in Kigali, Nyanza, and Rwamagana. The profile also identifies education, scientific and technological research, and community service as central institutional purposes [3].

For this study, Nyanza District is the application context, while UNILAK provides the academic setting in which the system is analysed and developed. The system is not claimed to be an officially commissioned district information system. It is an academic system designed to demonstrate how a district-level organization could centralize location monitoring.

## 3.3 Vision, mission and development objectives

The official Nyanza District profile presents the district as a historical and cultural centre and describes a development focus involving cultural tourism and sustainable development [1]. In the context of this study, the relevant operational vision is a district administration supported by timely, reliable, and accessible information for coordinating mobile activities.

The relevant mission for the proposed system is to support coordination and accountability by making recent location information available to authorized users through a centralized web interface. This system mission is consistent with the national transport emphasis on mobility, accessibility, reliability, safety, monitoring, and evaluation [2].

The development objectives of the proposed system are to reduce reliance on fragmented communication, centralize location records, improve visibility of active trackers, support trip documentation, and provide a technical foundation for future alerts and spatial analysis.

## 3.4 Description of existing system activities

### 3.4.1 Tracker registration

In the assumed existing arrangement, a new vehicle, phone, or field user is identified through a conversation between an administrator and a supervisor. Basic information may be written in a notebook, spreadsheet, or message thread. The assignment of the device to a user or route is then communicated informally.

This procedure can be adequate for a small number of users, but it does not create one authoritative tracker register. Names, device identifiers, assignments, and status changes may be stored in different places. Figure 3.1 shows the process model used for this study.

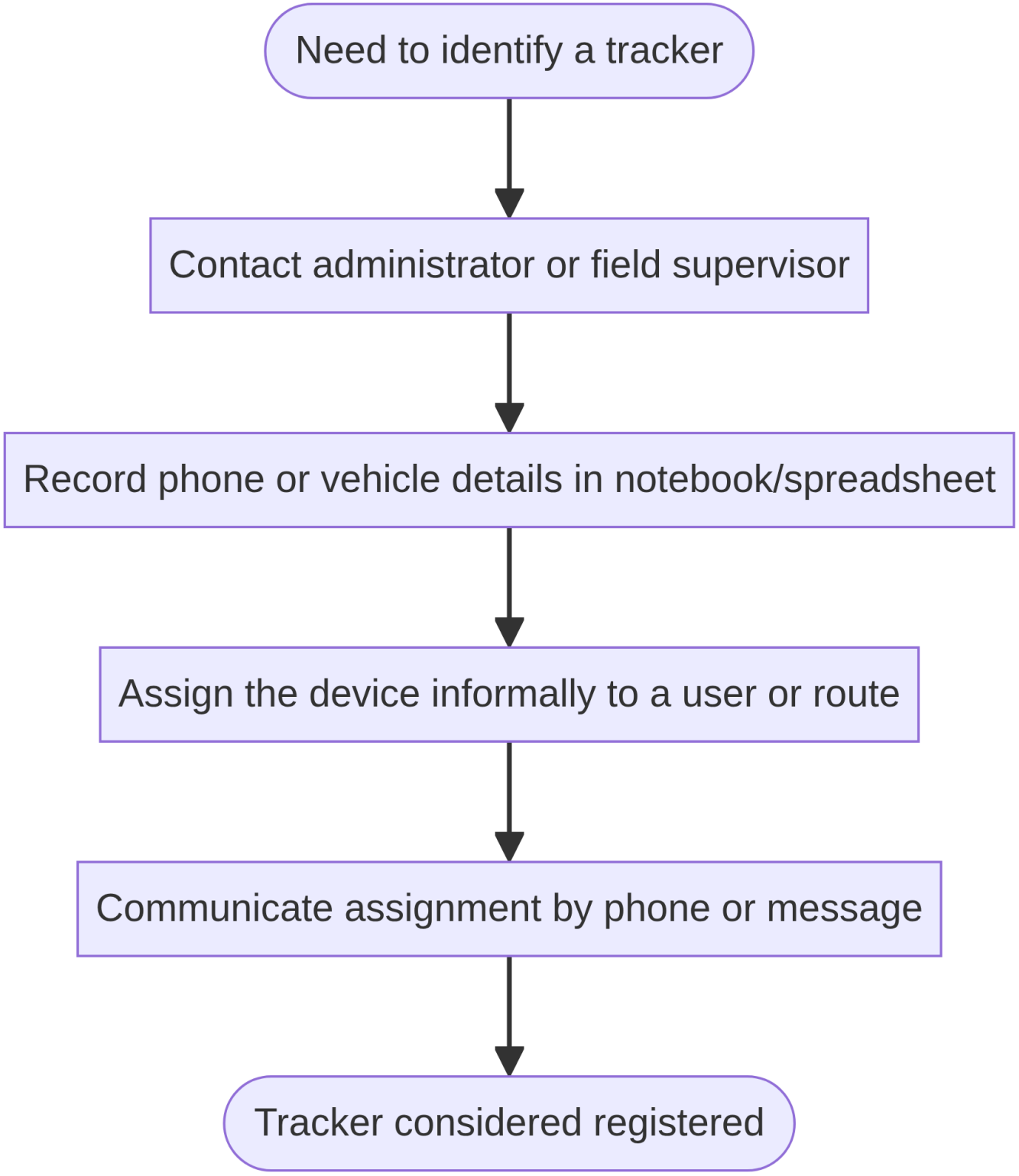



*Figure 3.1: Existing tracker registration process.*

### 3.4.2 Location reporting

In the assumed existing arrangement, a field user travels and communicates a location only when a supervisor requests an update or when the user considers it necessary. The supervisor records the

reported location and time manually. If no update is requested, the movement may not create a central record.

This procedure depends on availability of the user and supervisor, accuracy of verbal descriptions, and consistency of manual recording. Figure 3.2 presents the process.

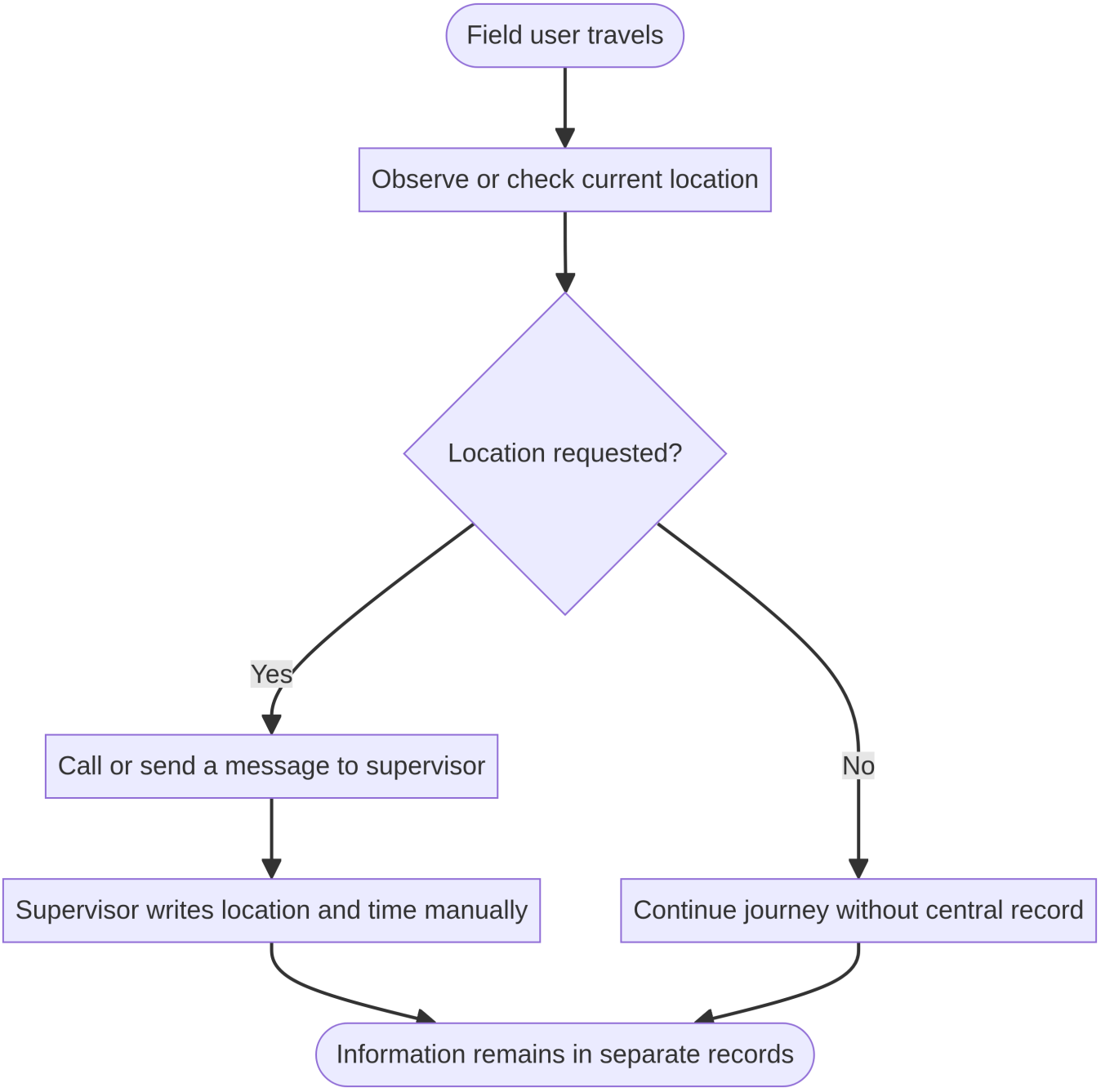



*Figure 3.2: Existing location reporting process.*

### 3.4.3 Trip monitoring

Trip monitoring in the assumed existing arrangement depends on a starting communication, periodic verbal updates, and follow-up when a delay or incident is reported. The supervisor does not

continuously see the position on a map. Monitoring therefore depends on the frequency and quality of communication.

Figure 3.3 illustrates this manual monitoring process. The repeated follow-up loop indicates why a centralized recent-location view could reduce unnecessary communication and improve situational awareness.

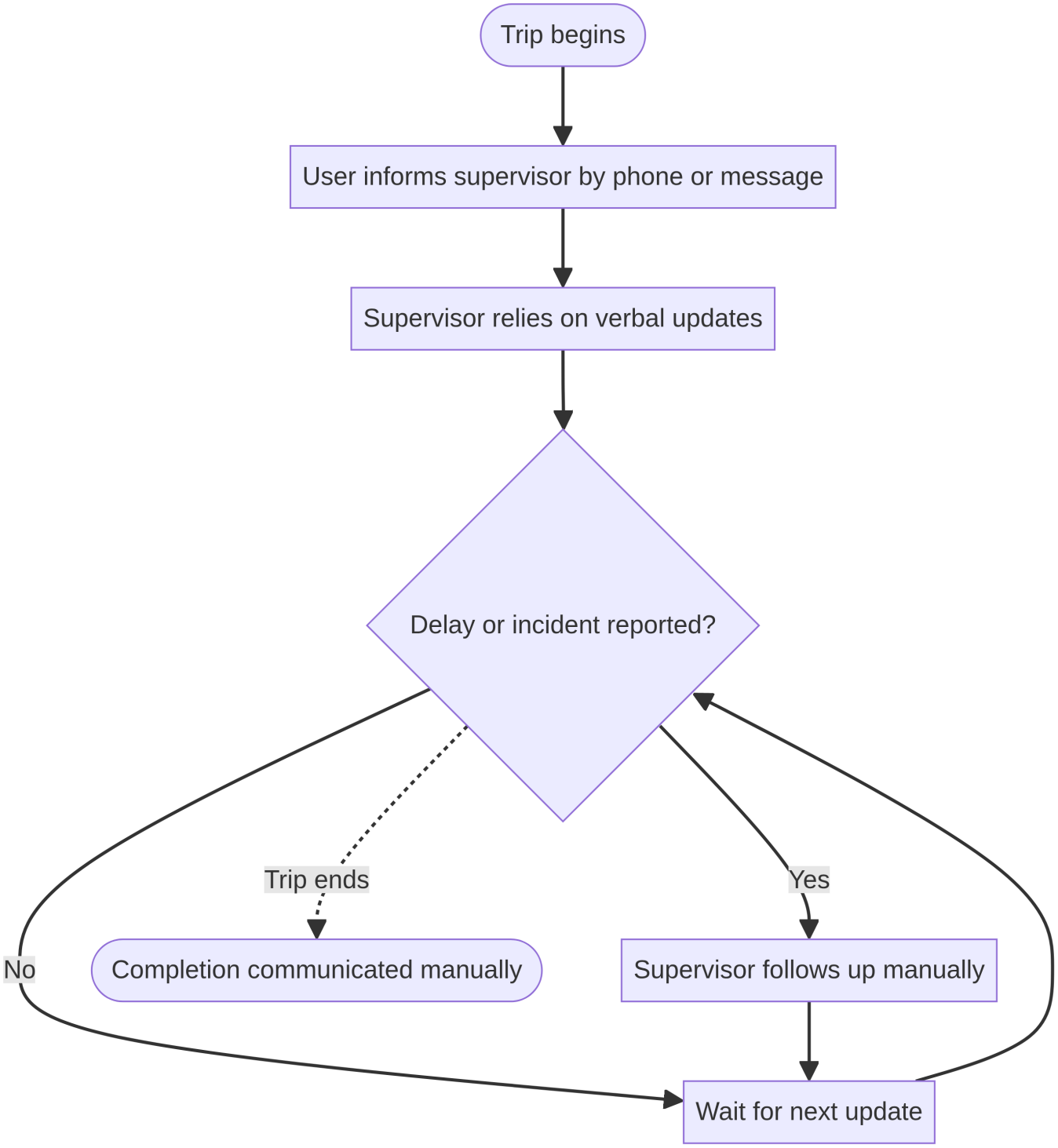



*Figure 3.3: Existing trip monitoring process.*

### 3.4.4 Trip reporting and record keeping

After a trip, the supervisor collects calls, messages, and paper notes and attempts to compare reported times and locations. A summary report may then be prepared and filed or shared with management. Because the information is collected from separate sources, historical analysis can be difficult and the completeness of the report depends on the quality of the records received.

Figure 3.4 shows the assumed process for trip reporting and record keeping.

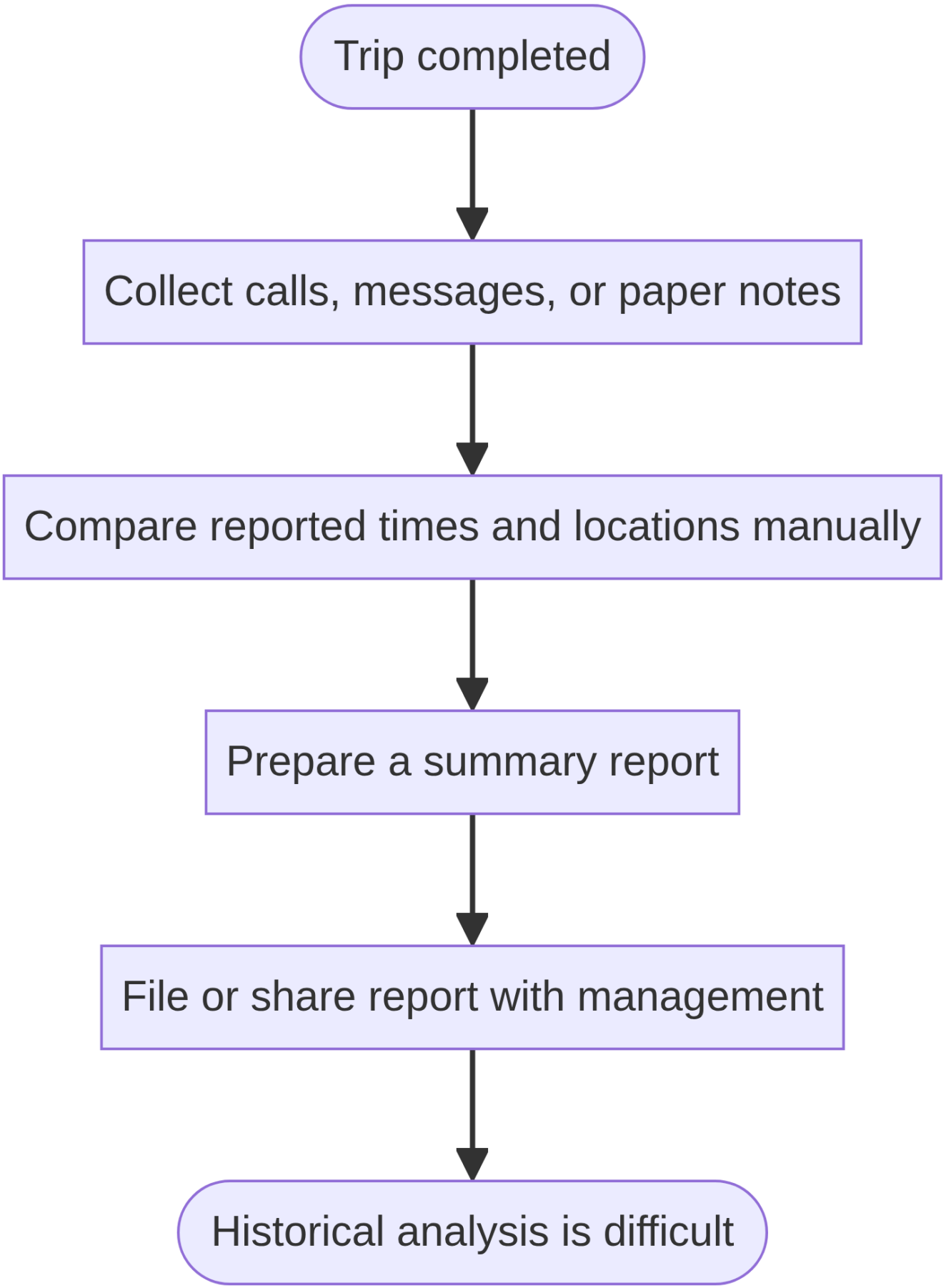

Trip completed
Collect calls, messages, or paper notes
Compare reported times and locations manually
Prepare a summary report
File or share report with management
Historical analysis is difficult

*Figure 3.4: Existing trip reporting and record-keeping process.*

## 3.5 Weaknesses of the existing system

The existing arrangement has limited centralization. Tracker information and location updates may be distributed across notebooks, spreadsheets, phone calls, and messaging platforms. This makes it difficult to maintain a single current register.

The arrangement also provides limited real-time visibility. A supervisor can know the location only after a user communicates it. If the user is unavailable, busy, or experiencing connectivity problems, the supervisor may not receive an update. The system therefore cannot reliably show the latest position of every mobile user.

Manual records can contain inconsistent names, missing timestamps, imprecise place descriptions, and duplicated entries. It is also difficult to draw a route from text messages or to calculate the distance and time travelled without additional processing.

The current arrangement does not automatically validate coordinates because it usually records descriptions rather than latitude and longitude. It also does not provide a standard method for showing whether a tracker is online, offline, or stale.

Finally, manual reporting increases administrative workload. A supervisor must request updates, transcribe information, compare records, and prepare summaries. The arrangement may also expose sensitive location information through uncontrolled communication channels.

## 3.6 Proposed solutions

The proposed system introduces a centralized tracker register containing the tracker name, type, status, and creation time. This replaces dispersed identification records with a database record that can be queried by the dashboard and API.

The smartphone tracker page uses browser geolocation to capture latitude, longitude, speed, accuracy, and time. It sends the observation to the server as a structured JSON request. The server validates the device and geographic ranges before storing the point in the locations table.

The dashboard provides a map-based view of latest locations and automatically refreshes recent device information. A manager can select a tracker to inspect its coordinates, speed, accuracy, last-seen time, and historical route. This reduces dependence on repeated phone calls and creates a consistent record for review.

The trip functions provide explicit start and end events. A trip is associated with the tracker and can be queried later. This creates a foundation for future reports, distance calculations, route-deviation alerts, and performance analysis.

The proposed system does not remove the need for human judgment or field communication. It provides a complementary digital record. In a production setting, it should be combined with user

consent, secure authentication, retention controls, and procedures for responding to inaccurate or unavailable location data.

# CHAPTER 4: ANALYSIS & DESIGN OF NEW SYSTEM

## 4.1 Introduction

This chapter presents the analysis and design of the proposed web-based GPS location tracking and management system. It describes the development methodology, modelling approach, tools, functions, system boundary, data flows, requirements, and database design. The design is based on a smartphone tracker, a web dashboard, an Express API, and a relational SQLite database.

## 4.2 Development methodologies, models and tools

### 4.2.1 Unified Process

The study adopts the Unified Process as a guiding software-development framework. The Unified Process is iterative and incremental. Its phases are inception, elaboration, construction, and transition. During inception, the problem and project scope are defined. During elaboration, requirements, architecture, and risks are analysed. During construction, the API, database, tracker interface, and dashboard are implemented. During transition, the system is tested, documented, and prepared for demonstration.

The use of iterations is appropriate because the project contains technical uncertainty around browser geolocation, network access, map rendering, and data persistence. A basic vertical slice can be built first: one smartphone submits one location and the dashboard displays it. Additional capabilities such as trips, history, and device registration can then be added without changing the entire design.

### 4.2.2 Unified Modeling Language

Unified Modeling Language (UML) is used to communicate the structure and behaviour of the system. The function diagram presents the main functional decomposition. The context diagram presents external actors and the system boundary. The data-flow diagram shows movement of information between processes and data stores. The ERD presents the database entities and their relationships. Together, the models create a traceable connection between requirements, implementation, and data storage.

### 4.2.3 Object-Oriented Methodology

Object-Oriented Methodology (OOM) analyses a system in terms of objects, their data, and their responsibilities. In this system, important objects include Device, Location, Trip, TrackerClient, Dashboard, and ApiService. A Device has an identity and status. A Location belongs to a Device and records a geospatial observation. A Trip belongs to a Device and represents a period of movement. The object-oriented perspective supports modular code and makes it easier to extend the system with users, geofences, alerts, or maintenance records.

### 4.2.4 Development tools

The client interface uses HTML, CSS, and JavaScript. The server uses Node.js and Express.js. SQLite is used as the relational database for the implemented system. Leaflet.js renders interactive map markers and route lines, while OpenStreetMap provides map tiles. Git and GitHub provide version control. Mermaid source files are used for diagrams, and the system can be run with `npm install` followed by `npm start`.

## 4.3 Function diagram

The function diagram decomposes the proposed system into tracker management, location tracking, trip management, and monitoring/reporting. Tracker management registers and selects devices. Location tracking requests permission, captures coordinates, validates observations, and stores them. Trip management creates and completes trip records. Monitoring and reporting provide the map, telemetry, and route history used by the administrator.

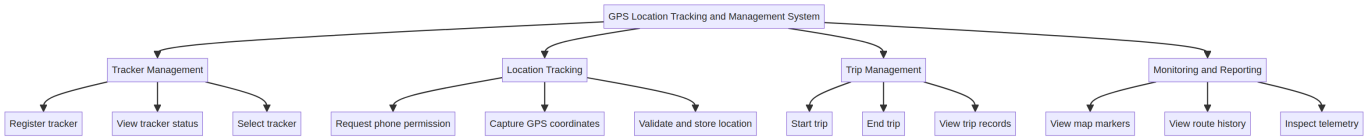



*Figure 4.1: Function diagram of the proposed GPS location tracking and management system.*

## 4.4 Context diagram

The context diagram shows the proposed system as one central process and identifies the two primary external actors. The administrator or fleet manager sends tracker and monitoring requests and receives map, status, telemetry, and history information. The smartphone tracker user grants permission, starts or stops tracking, and starts or ends trips. The system receives location data from the tracker and provides stored information to the administrator.

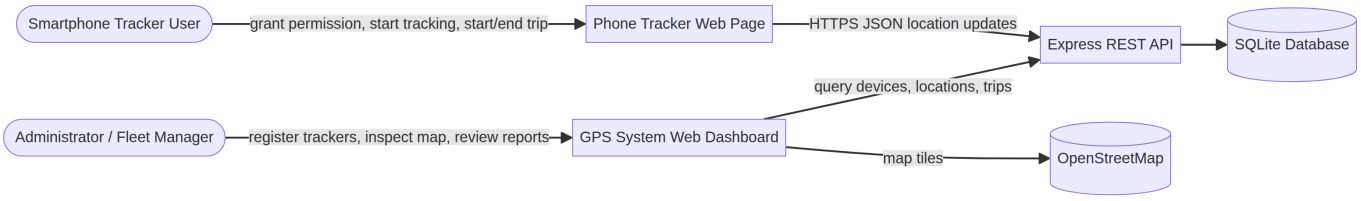



*Figure 4.2: Context diagram of the proposed GPS location tracking and management system.*

## 4.5 Data flow diagram (level 0)

The level 0 data-flow diagram decomposes the system into tracker management, location capture, and fleet monitoring. The administrator interacts with tracker management and receives monitoring outputs. The smartphone tracker provides coordinates and trip actions to the location-capture process. Validated location points are stored in the locations data store, while device and trip information are stored in their respective data stores.

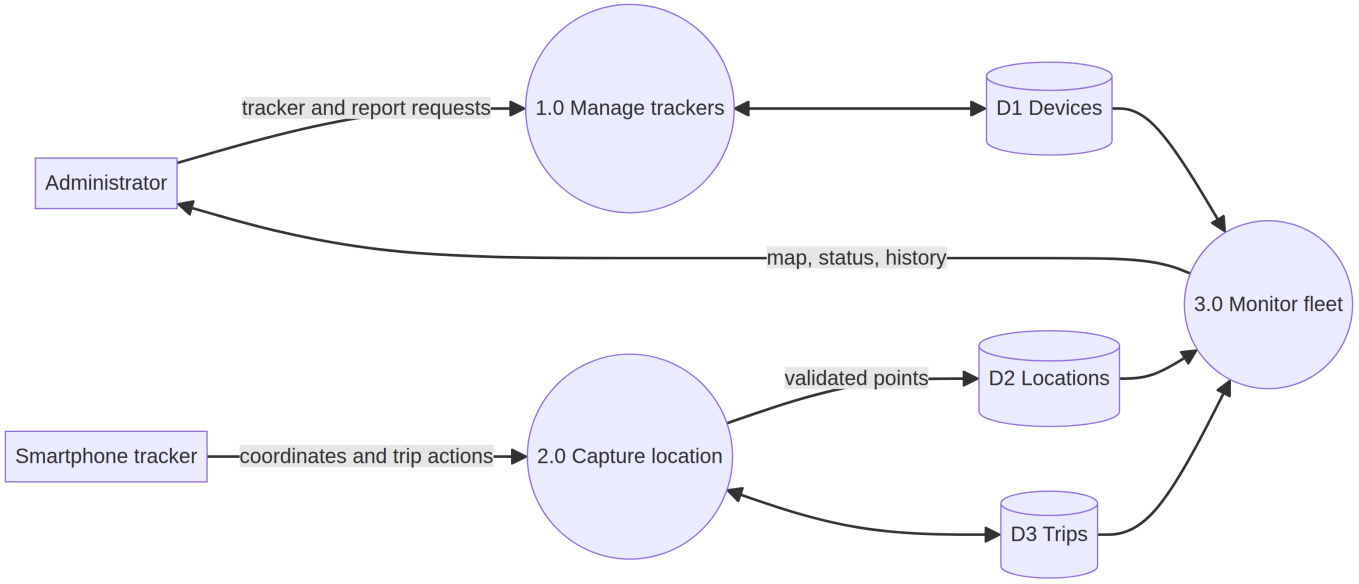



*Figure 4.3: Level 0 data-flow diagram of the proposed GPS location tracking and management system.*

## 4.6 Functional and non-functional requirements

### 4.6.1 Functional requirements

| Requirement ID | Functional requirement |
|---|---|
| FR-01 | The system shall allow an administrator to register a tracker with a name and device type. |
| FR-02 | The system shall list all registered trackers. |
| FR-03 | The phone tracker shall request browser location permission before tracking. |
| FR-04 | The phone tracker shall capture latitude, longitude, speed, accuracy, and time. |
| FR-05 | The system shall submit location observations to the server through a REST API. |
| FR-06 | The server shall validate the tracker identifier and coordinate ranges. |
| FR-07 | The server shall store valid location observations in the database. |
| FR-08 | The dashboard shall display latest tracker positions on an interactive map. |
| FR-09 | The dashboard shall display tracker status and last-seen time. |
| FR-10 | The administrator shall view the history of a selected tracker. |
| FR-11 | The user shall start a trip for a tracker. |
| FR-12 | The user shall end an active trip. |
| FR-13 | The administrator shall view recorded trip information. |

### 4.6.2 Non-functional requirements

| Requirement ID | Non-functional requirement |
|---|---|
| NFR-01 | The system shall provide a responsive interface usable on a laptop and smartphone. |
| NFR-02 | The API shall reject invalid latitude and longitude values. |
| NFR-03 | The system shall preserve location and trip records after a server restart. |
| NFR-04 | The dashboard shall refresh recent device information automatically. |
| NFR-05 | The interface shall display understandable errors for denied permission and failed requests. |
| NFR-06 | The deployment shall use HTTPS when accessed from a physical phone outside localhost. |
| NFR-07 | The system should use parameterized database statements to reduce injection risk. |
| NFR-08 | Production deployment shall restrict access to authorized users and protect personal location data. |
| NFR-09 | The system should remain sufficiently simple for installation and demonstration in a BSc environment. |

## 4.7 Database design

### 4.7.1 Entities

The database contains three main entities. The **Device** entity represents a registered smartphone or future GPS device. The **Location** entity represents one position observation submitted by a device. The **Trip** entity represents a period of movement associated with a device.

The design intentionally separates devices from locations. A device can submit many observations over time, while each observation belongs to one device. This structure avoids repeating device names and types in every location record. Trips are also separated because a device may complete many trips, and a trip may be analysed independently of the device’s entire history.

### 4.7.2 Relationships and cardinality

The relationship between Device and Location is one-to-many. One device can produce zero, one, or many location records. Every location record must belong to exactly one device. The relationship between Device and Trip is also one-to-many. One device can have zero, one, or many trips, and every trip must belong to exactly one device.

The foreign key `locations.device_id` references `devices.id`. The foreign key `trips.device_id` references `devices.id`. Cascade deletion is used in the implemented system so that deleting a device also removes dependent location and trip records. A production system should apply deletion policies carefully because location history may be subject to legal, operational, or audit requirements.

### 4.7.3 Attributes

A Device has `id`, `name`, `device_type`, `status`, and `created_at`. A Location has `id`, `device_id`, `latitude`, `longitude`, `speed`, `accuracy`, and `recorded_at`. A Trip has `id`, `device_id`, `started_at`, `ended_at`, and `status`.

The `id` fields uniquely identify records. `latitude` and `longitude` represent position. `speed` represents speed in kilometres per hour in the client payload. `accuracy` represents the approximate accuracy reported by the browser in metres. ISO-formatted timestamps provide a consistent representation of time.

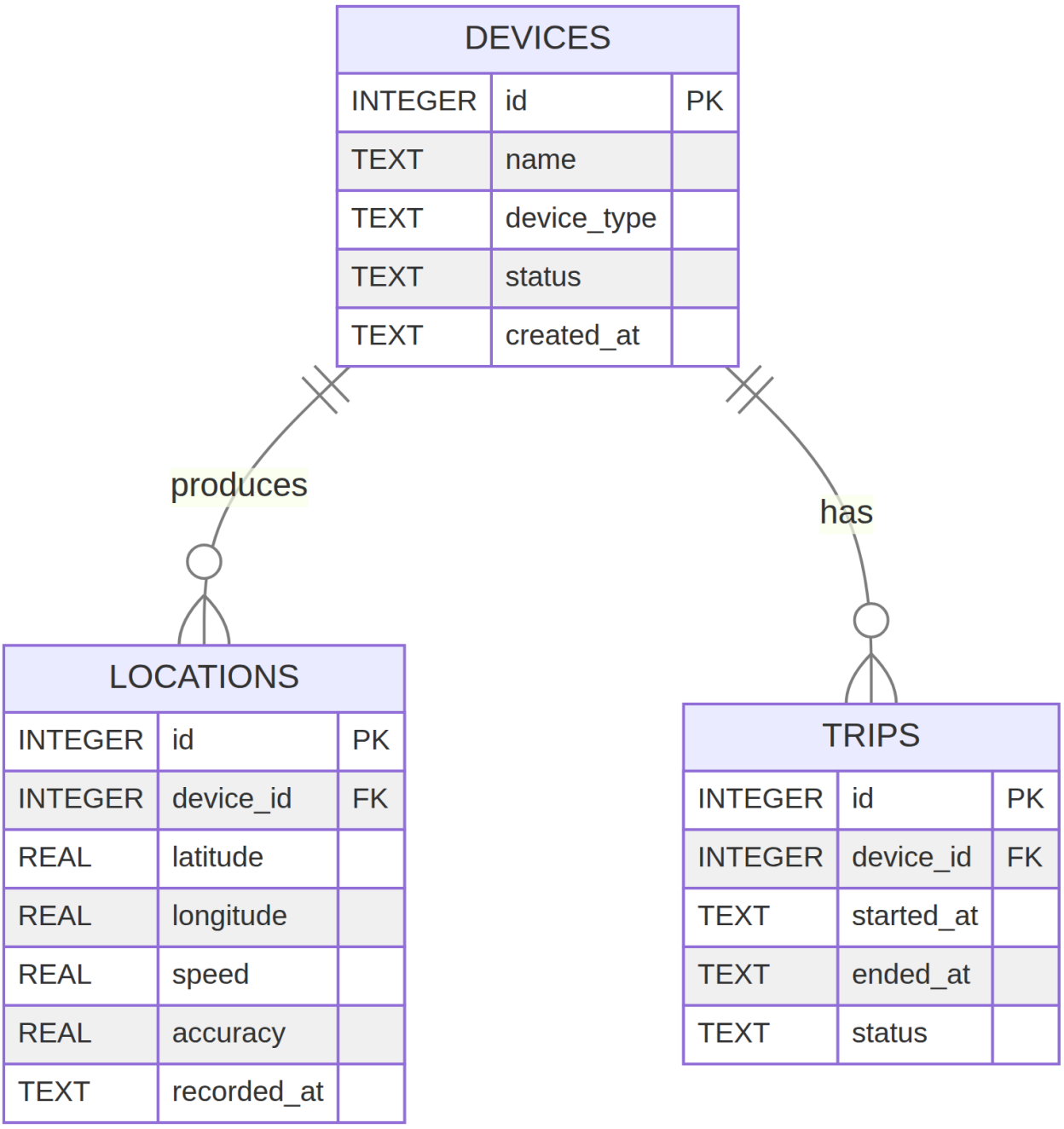



*Figure 4.4: Entity relationship diagram of the proposed GPS location tracking and management system.*

### 4.7.4 Physical model

The physical model is implemented in SQLite. The database contains the tables `devices`, `locations`, and `trips`. The `devices`, `locations`, and `trips` identifiers are integer primary keys generated automatically. The fields that store names, types, statuses, and timestamps use text values. Geographic measurements use real-number fields.

The physical schema also creates an index on `(device_id, recorded_at DESC)` in the locations table. This index supports the common query that retrieves recent location points for a selected device. Foreign-key enforcement is enabled. The standalone implementation schema is included in `docs/database-schema.sql`.

```
CREATE TABLE devices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  device_type TEXT NOT NULL DEFAULT 'smartphone',
  status TEXT NOT NULL DEFAULT 'offline',
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE locations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  latitude REAL NOT NULL,
  longitude REAL NOT NULL,
  speed REAL DEFAULT 0,
  accuracy REAL,
  recorded_at TEXT NOT NULL,
  FOREIGN KEY (device_id) REFERENCES devices(id) ON DELETE CASCADE
);

CREATE TABLE trips (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  started_at TEXT NOT NULL,
  ended_at TEXT,
  status TEXT NOT NULL DEFAULT 'active',
  FOREIGN KEY (device_id) REFERENCES devices(id) ON DELETE CASCADE
);
```

### 4.7.5 Logical model

The logical model can be represented as follows:

`DEVICES (id, name, device_type, status, created_at)`

`LOCATIONS (id, device_id, latitude, longitude, speed, accuracy, recorded_at)`

`TRIPS (id, device_id, started_at, ended_at, status)`

The primary key of `DEVICES` is `id`. The primary key of `LOCATIONS` is `id`, and `device_id` is a foreign key referencing `DEVICES.id`. The primary key of `TRIPS` is `id`, and `device_id` is a

foreign key referencing `DEVICES.id`. The logical model is normalized sufficiently for the implemented system because device details are not repeated in location or trip rows and each field stores one principal value.

### 4.7.6 Data dictionary

| Entity | Attribute | Data type | Key or rule | Description |
|---|---|---|---|---|
| Device | id | INTEGER | Primary key, auto-increment | Unique identifier of a tracker |
| Device | name | TEXT | Required | Human-readable tracker name |
| Device | device_type | TEXT | Default: smartphone | Type of tracking device |
| Device | status | TEXT | Default: offline | Current derived status |
| Device | created_at | TEXT | Default timestamp | Time at which tracker was registered |
| Location | id | INTEGER | Primary key, auto-increment | Unique location record |
| Location | device_id | INTEGER | Foreign key, required | Device that submitted the point |
| Location | latitude | REAL | -90 to 90 | North/south coordinate |
| Location | longitude | REAL | -180 to 180 | East/west coordinate |
| Location | speed | REAL | Default: 0 | Speed associated with the point in km/h |
| Location | accuracy | REAL | Optional | Estimated GPS accuracy in metres |
| Location | recorded_at | TEXT | Required timestamp | Time at which the point was recorded |
| Trip | id | INTEGER | Primary key, auto-increment | Unique trip record |
| Trip | device_id | INTEGER | Foreign key, required | Device assigned to the trip |
| Trip | started_at | TEXT | Required timestamp | Time at which the trip began |

| Entity | Attribute | Data type | Key or rule | Description |
|---|---|---|---|---|
| Trip | ended_at | TEXT | Optional | Time at which the trip ended |
| Trip | status | TEXT | Active or completed | Current state of the trip |

**Sources cited in this draft:** [1] Nyanza District, “About Nyanza District,” official district website, https://www.nyanza.gov.rw/en/district/about-the-district. [2] Ministry of Infrastructure, Rwanda, “Transport,” https://www.mininfra.gov.rw/1/transport. [3] University of Lay Adventists of Kigali, “UNILAK Profile,” https://site.unilak.ac.rw/elementor-1875/.

# CHAPTER 5: IMPLEMENTATION OF THE NEW SYSTEM

## 5.1 Introduction

This chapter describes the implementation of the new web-based GPS location tracking and management system. The implementation translates the requirements and designs presented in Chapter Four into a working application that can be run through a web browser. The system is implemented as a client-server information system comprising a management dashboard, a smartphone tracker interface, an application server, a REST API, and a relational database.

The implementation was guided by the principle of keeping the system accessible and practical for Nyanza District operations. A smartphone is used as the GPS tracking device, while the manager uses a computer or another browser-enabled device to monitor registered trackers. This arrangement allows the system to be used without purchasing a dedicated GPS unit or owning a car during demonstrations and normal operational use.

## 5.2 Implementation environment

The system was implemented using the following software and technologies:

| Component | Technology used | Implementation purpose |
| --- | --- | --- |
| Client interface | HTML5, CSS3 and JavaScript | Dashboard and smartphone tracker pages |
| Server runtime | Node.js | Execution environment for the application server |
| Web framework | Express.js | HTTP server, routing and REST API |
| Database | SQLite | Persistent storage of devices, locations and trips |
| Mapping library | Leaflet.js | Interactive map markers and route visualization |
| Map data | OpenStreetMap | Base map tiles for the monitoring dashboard |
| Data format | JSON | Communication between browser clients and the REST API |
| Version control | Git and GitHub | Source-code management and release storage |

The application is started using the Node Package Manager scripts defined in `package.json`. The server listens on the configured application port and serves the dashboard, phone tracker page, stylesheets, client scripts, route manifest, and REST API from one application origin.

The source code, implementation documentation, and release package are maintained in the GitHub repository at https://github.com/MiracleCodez/gps-system. The application can be operated locally for development or deployed to a managed web host that supports a Node.js server and persistent database storage.

## 5.3 Server-side implementation

The server-side implementation is contained in `server.js`. Express is configured to serve the static files in the `public` directory and to expose the REST endpoints required by the two browser interfaces. The server also initializes the SQLite database when the application starts. Database tables are created only when they do not already exist, which allows existing records to remain available after a restart.

The server performs four main responsibilities. First, it serves the web pages and static client resources. Second, it receives and validates API requests. Third, it writes and reads devices,

locations, and trips in SQLite. Fourth, it provides JSON responses that allow the dashboard and phone tracker to update their interfaces.

The server uses parameterized SQL statements for database operations. This prevents user-provided values from being inserted directly into SQL command strings. It also validates required fields, checks that a referenced tracker exists, and verifies that latitude and longitude values are within valid geographic ranges.

## 5.4 REST API implementation

The REST API provides a communication layer between the browser clients and the server. The implemented endpoints are as follows:

| Method | Endpoint | Implementation function |
|---|---|---|
| GET | `/api/health` | Reports whether the server is available |
| GET | `/api/devices` | Returns registered trackers and their latest location information |
| POST | `/api/devices` | Creates a new tracker record |
| GET | `/api/devices/:id/locations` | Returns ordered location history for one tracker |
| POST | `/api/locations` | Validates and stores a GPS location observation |
| POST | `/api/trips/start` | Creates an active trip for a tracker |
| POST | `/api/trips/end` | Completes the active trip for a tracker |
| GET | `/api/trips` | Returns recent trip records |

When a phone submits a location, the request contains a device identifier, latitude, longitude, speed, accuracy, and recorded time. The server verifies the device identifier and coordinate values before storing the observation. A successful request returns a JSON response confirming that the location was recorded.

The device-list endpoint returns the registered devices together with recent status and latest-location information. The dashboard uses this endpoint to populate the tracker list, map markers, status labels, and telemetry panel. The location-history endpoint returns records in time order so that the client can display a route line and a history table.

Trip endpoints provide a clear beginning and ending for a movement activity. The start endpoint creates a record with active status. The end endpoint records the end time and changes the status to completed. This implementation allows management records to distinguish an active journey from a completed journey.

## 5.5 Database implementation

The database implementation uses SQLite and consists of three related tables: `devices`, `locations`, and `trips`. The `devices` table stores the identity and status of each tracker. The `locations` table stores coordinate observations. The `trips` table stores journey periods.

The one-to-many relationships defined in Chapter Four are implemented through foreign keys. A device can have many location observations and many trips. Each location and trip belongs to one device. An index on device identifier and recorded time supports the retrieval of recent location history.

The database is suitable for the current system because it is lightweight, easy to install, and able to preserve records in a local database file. For an expanded multi-department deployment, the same logical model can be migrated to a server database such as PostgreSQL with spatial extensions. Such a migration would preserve the main entities while providing stronger concurrency and spatial-query capabilities.

## 5.6 Dashboard implementation

The management dashboard is implemented in `public/index.html`, `public/app.js`, and `public/styles.css`. It is designed for the administrator or fleet manager. The dashboard contains the following functional areas:

1. **Tracker registration:** A manager enters a tracker name and device type. The form sends a request to the device-creation endpoint.
2. **Tracker list:** Registered trackers are displayed with their current status and most recent activity.
3. **Interactive map:** Latest tracker locations are represented as map markers. Selecting a tracker updates the map and telemetry panel.
4. **Telemetry panel:** The dashboard displays latitude, longitude, speed, accuracy, and last-seen time for the selected tracker.
5. **Route history:** Stored location points are requested from the server and represented as a route line and history records.
6. **Trip information:** The dashboard provides access to recorded trips and their active or completed status.

The dashboard refreshes recent device information periodically. This allows a manager to view new location submissions without repeatedly refreshing the entire page. Error messages are displayed when the server cannot be reached or when an API request is rejected.

## 5.7 Smartphone tracker implementation

The smartphone tracker is implemented in `public/tracker.html`, `public/tracker.js`, and the shared stylesheet. A user opens the tracker page on a smartphone and selects a registered tracker. When tracking is started, the browser requests permission to access location information through the Geolocation API.

After permission is granted, the browser watches for position changes. Each position object is converted into the system's location payload. The implementation extracts latitude, longitude, speed, accuracy, and time, then sends the payload to `/api/locations`. The page reports the latest coordinate, estimated accuracy, and number of successful updates so that the user can see whether tracking is active.

The same page provides start-trip and end-trip controls. Starting a trip sends the selected device identifier to the trip-start endpoint. Ending a trip sends the identifier to the trip-end endpoint. The interface informs the user when permission is denied, when the phone cannot obtain a position, or when the server cannot receive an update.

For normal use, the phone should remain connected to the internet and the tracker page should remain active. The browser must be allowed to use location services. When using a physical phone outside localhost, the deployed website should use HTTPS because modern browsers restrict geolocation on insecure origins.

## 5.8 Map and route implementation

The mapping component uses Leaflet to create an interactive map. OpenStreetMap tiles provide the background map. When the dashboard receives a latest location, it creates or updates a marker for the selected tracker. The marker popup identifies the tracker and provides its latest position information.

For route history, the dashboard converts the ordered location records into coordinate pairs. Leaflet then draws a polyline through those points. The map is fitted to the available route so that the manager can view the movement area. The route is based on stored GPS observations and therefore becomes more detailed as the tracker submits additional points.

The map implementation separates presentation from storage. Leaflet and OpenStreetMap display the data, while the server and SQLite database remain responsible for preserving the location records. This separation allows a future mapping library or spatial database to be introduced without changing the fundamental tracker workflow.

## 5.9 Application workflow

The complete operational workflow is implemented as follows:

1. The manager opens the dashboard and registers a tracker.
2. The system stores the tracker in the devices table.
3. The user opens the phone tracker page and selects the registered tracker.
4. The user grants browser location permission and starts tracking.
5. The smartphone obtains a position and sends a JSON location request.
6. The server validates the device and coordinates.
7. The server stores the location in the locations table and updates the tracker activity.
8. The dashboard refreshes and displays the latest marker and telemetry.
9. The user starts a trip when a journey begins and ends it when the journey is complete.
10. The manager selects a tracker to view its route history and trip information.

This workflow connects the requirements, system diagrams, database model, and user interfaces. It also allows the system to be used with a walking smartphone user when a vehicle is not available.

## 5.10 Deployment and operation

The application is maintained in the GitHub repository `MiracleCodez/gps-system`. For local operation, the user installs the Node.js dependencies, starts the server, and opens the dashboard in a browser. For online operation, the project requires a hosting environment that supports a Node.js server, environment configuration, public HTTPS access, and persistent database storage.

The operational URL should be used for the dashboard and the `/tracker.html` page. The manager shares the tracker page URL with an authorized phone user. The phone user must grant location permission and keep the page available during tracking. The manager uses the dashboard URL to monitor the latest location and review the stored history.

Deployment should include a health endpoint, server logs, database backup, and a documented restart procedure. The system should also be protected through HTTPS and access restrictions before real personal or vehicle location data is collected. These operational controls support the use of the implemented system while acknowledging the responsibilities associated with location information.

## 5.11 Implementation summary

The new system has been implemented as a working web application with a dashboard, smartphone tracker page, REST API, relational database, map visualization, route history, and trip

management. The implementation directly reflects the design presented in Chapter Four. It can be used with a smartphone in place of a dedicated GPS device and can be expanded later with authentication, roles, geofencing, alerts, offline buffering, and advanced spatial analysis.

# CHAPTER 6: REFERENCES


[1] Nyanza District, “About Nyanza District,” Official Website of Nyanza District, Rwanda. Available at: https://www.nyanza.gov.rw/en/district/about-the-district. Accessed: 10 October 2026.

[2] Ministry of Infrastructure, Rwanda, “Transport,” Official Website of the Ministry of Infrastructure. Available at: https://www.mininfra.gov.rw/1/transport. Accessed: 10 October 2026.

[3] University of Lay Adventists of Kigali, “UNILAK Profile,” Official UNILAK Website. Available at: https://site.unilak.ac.rw/elementor-1875/. Accessed: 10 October 2026.

[4] OpenStreetMap Foundation, “OpenStreetMap,” Available at: https://www.openstreetmap.org/. Accessed: 10 October 2026.

[5] Leaflet contributors, “Leaflet: An Open-Source JavaScript Library for Interactive Maps,” Available at: https://leafletjs.com/. Accessed: 10 October 2026.

[6] Mozilla Developer Network, “Geolocation API,” Available at: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API. Accessed: 10 October 2026.

[7] MiracleCodez, “GPS System,” GitHub repository. Available at: https://github.com/MiracleCodez/gps-system. Accessed: 13 October 2026.